

Chapter 15. Volumes

When container images are instantiated as containers, the container needs context—context to CPU, memory, and I/O resources. Pods provide the network and the filesystem context for the containers within. The network is provided as the Pod's virtual IP address, and the filesystem is mounted to the hosting node's filesystem.

Applications running in the container can interact with the filesystem as part of the Pod context. A container's temporary filesystem is isolated from any other container or Pod and is not persisted beyond a Pod restart.

Essentially, a *volume* is a directory that's shareable between multiple containers of a Pod. [Figure 15-1](#) illustrates the differences between the temporary filesystem of a container and the use of a volume.

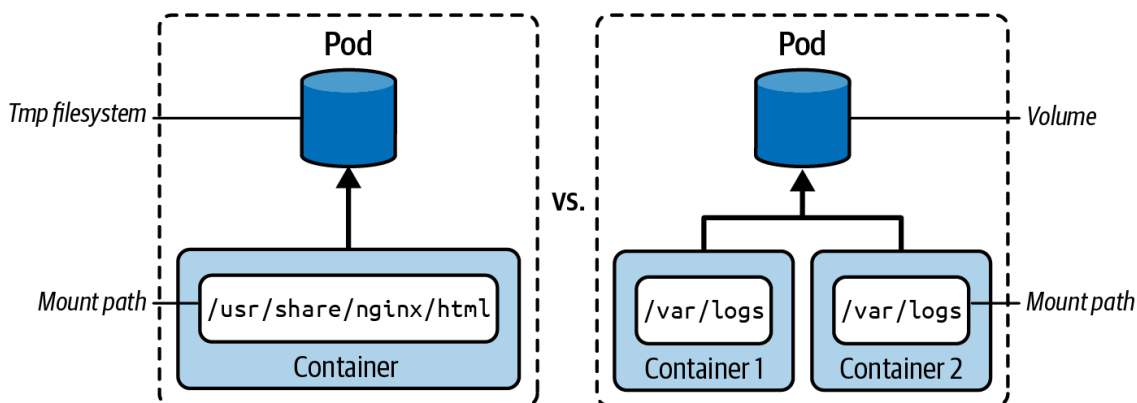


Figure 15-1. A container using the temporary filesystem versus a volume

In this chapter, you will learn about the different volume types and the process for defining and mounting a volume in a container.

The curriculum doesn't explicitly mention coverage of volume basics. However, you will definitely need to understand this concept for persistent volumes described in the next chapter.

Purpose of Volumes

In a nutshell, the Kubernetes *volume* concept aims to fulfill the goals described here:

Data persistence

Applications running in a container can use the temporary filesystem to read and write files. In the case of a container crash or a cluster/node restart, the kubelet will restart the container. Any data that had been written to the temporary filesystem is lost and cannot be retrieved anymore. The container effectively starts with a clean slate again. Volumes can provide persistent storage that survives container restarts, ensuring important data isn't lost.

Data sharing

There are many use cases for wanting to mount a volume in a container. One of the most prominent use cases is [multi-container Pods](#) that use a volume to exchange data between a main application container and a sidecar, enabling them to share files and communicate through the filesystem.

Decoupling storage from containers

Volumes abstract storage details from the application, allowing you to change storage backends without modifying container images.

Volume Types

Every volume needs to define a type. The type determines the medium that backs the volume and its runtime behavior. The [Kubernetes documentation](#) offers a long list of volume types. Some of the types—for exam-

ple, `azureDisk`, `awsElasticBlockStore`, or `gcePersistentDisk` — are available only when running the Kubernetes cluster in a specific cloud provider. Many of those cloud-provider-specific volume types have already been deprecated.

[Table 15-1](#) shows a reduced list of volume types that I deem to be most relevant to the exam.

Table 15-1. Volume types relevant to exam

Type	Description
<code>emptyDir</code>	Empty directory in Pod with read/write access. Persisted for only the lifespan of a Pod. A good choice for cache implementations or data exchange between containers within the same Pod.
<code>hostPath</code>	File or directory from the host node's filesystem.
<code>configMap</code> , <code>secret</code>	Provides a way to inject configuration data. For practical examples, see Chapter 10 .
<code>nfs</code>	An existing Network File System (NFS) share. Preserves data after Pod restart.
<code>persistentVolumeClaim</code>	Claims a persistent volume. For more information, see “Creating PersistentVolumeClaims” .

Creating and Accessing Volumes

Defining a volume for a Pod requires two steps. First, you need to declare the volume itself using the attribute `spec.volumes[ ]`. As part of the definition, you provide the name and the type. Just declaring the volume won't be sufficient, though. Second, the volume also needs to be mounted to a path of the consuming container via `spec.containers[ ].volumeMounts[ ]`. The mapping between the volume and the volume mount occurs by the matching name.

From the YAML manifest stored in the file *pod-with-volume.yaml* and shown in [Example 15-1](#), you can see the definition of a volume with type `emptyDir`. The volume has been mounted to the path

`/usr/share/nginx/html` inside of the container named `nginx` and to the path `/data` inside of the container named `sidecar`.

Example 15-1. A Pod defining and mounting a volume

```
apiVersion: v1
kind: Pod
metadata:
  name: business-app
spec:
  volumes:
    - name: shared-data
      emptyDir: {}
  containers:
    - name: nginx
      image: nginx:1.27.1
      volumeMounts:
        - name: shared-data
          mountPath: /usr/share/nginx/html
    - name: sidecar
      image: busybox:1.37.0
      volumeMounts:
        - name: shared-data
          mountPath: /data
```

- ❶ Specifies a volume of type `emptyDir`. The curly braces mean that we don't want to provide additional configuration, e.g., a size limit.
- ❷ Mounts the volume to containers with different mount paths inside of the container.

Let's create the Pod from the YAML definition above. The Pod runs two containers:

```
$ kubectl apply -f pod-with-volume.yaml
pod/business-app created
$ kubectl get pod business-app
```

NAME	READY	STATUS	RESTARTS	AGE
business-app	2/2	Running	0	43s

The following commands open an interactive shell to the container named `nginx` after the Pod's creation and then navigates to the mount path. You can see that the volume type `emptyDir` initializes the mount path as an empty directory:

```
$ kubectl exec business-app -it -c nginx -- /bin/sh
# cd /usr/share/nginx/html
# pwd
/usr/share/nginx/html
# ls
# touch example.html
# ls
example.html
```

New files and directories can be created as needed without limitations.

Read-Only Volume Mounts

Some data is only meant for consumption, for example, configuration data provided through a volume. You can mark a volume mount to be read-only. Kubernetes will prevent any write operation on that volume mount. It's important to understand that other containers may use the same volume in read/write mode.

To make a volume mount read-only, assign the value `true` to the attribute `spec.containers[ ].volumeMounts[ ].readOnly`, as shown in [Example 15-2](#).

Example 15-2. A mount path marked as read-only

```
apiVersion: v1
kind: Pod
metadata:
  name: business-app
spec:
  volumes:
    - name: shared-data
      emptyDir: {}
  containers:
    - name: nginx
```

```
image: nginx:1.27.1
volumeMounts:
- name: shared-data
  mountPath: /usr/share/nginx/html
  readOnly: true
```

❶

❶ Prevents write operations for this mount path

Read-only mounts are not recursively read-only. You can force the behavior though by setting the attribute

`.spec.containers[ ].volumeMounts[ ].recursiveReadOnly` to `true`. See the [Kubernetes blog](#) for more information.

Summary

Kubernetes provides volumes to decouple storage from the container's lifecycle, enabling both ephemeral and persistent data storage, as well as data sharing between containers in a Pod.

Exam Essentials

Understand the need and use cases for a volume

Many production-ready application stacks running in a cloud native environment need to persist data. Read up on common use cases and explore recipes that describe typical scenarios. You can find some examples in the O'Reilly books [Kubernetes Best Practices](#) by Brendan Burns et al. and [Cloud Native DevOps with Kubernetes](#) by John Arundel and Justin Domingus.

Practice defining and consuming volumes

Volumes are a cross-cutting concept applied in different areas of the exam. Know where to find the relevant documentation for defining a volume and the multitude of ways to consume a volume from a container. Definitely revisit [Chapter 10](#) for a deep dive on how to mount ConfigMaps and Secrets as a volume.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Create a Pod YAML manifest with two containers that use the image `alpine:3.22.2`. Provide a command for both containers that keeps them running forever.

Define a volume of type `emptyDir` for the Pod. Container 1 should mount the volume to path `/etc/a`, and Container 2 should mount the volume to path `/etc/b`.

Open an interactive shell for Container 1 and create the directory *data* in the mount path. Navigate to the directory and create the file *hello.txt* with the contents “Hello World.” Exit out of the container.

Open an interactive shell for Container 2 and navigate to the directory `/etc/b/data`. Inspect the contents of file *hello.txt*. Exit out of the container.